AXUR An Infoblox Company

TLP:CLEAR MAY 2026

A MANIFESTO FROM INSIDE THE SHIFT

# Restructuring Knowledge Work for Human-Agent Teams

Seven weeks, twenty people, eight functions — and the bet that the layer under the agents is the asset that compounds.


AUTHORS

Fábio Ramos, CEO
Jônadas Techio, Staff PM

ORGANIZATION

Axur


COMPANION

*An Agentic Operating System — Seven Weeks of Practice*

Before Block published "From Hierarchy to Intelligence", we were already building the layer they describe. Over seven weeks, more than twenty people across eight functions inside Axur built a new operational playbook from the ground up. None of them were engineers. Building infrastructure was not in any of their job descriptions. We did not plan for it to happen this way. We are publishing this because every knowledge company is about to face the question we faced, and most are about to answer it wrong.

The wrong answer is some version of *buy more AI*. That answer creates a one-time productivity bump that everyone will call transformation. The companies that compound over the next decade will not be the ones that adopted the most agentic tools. They will be the ones that rebuilt the layer those tools read.

Two things made this possible in the last eighteen months. The cost of execution dropped sharply: as Ian Beacraft put it earlier this year, building a working prototype with AI now costs less than holding the meeting that would plan it. And the format the agents read standardized. Markdown in Git, with a thin instruction manifest at the root, modular references loaded on demand. The same pattern shows up in Claude Code, Cursor, GitHub Copilot, in the AGENTS.md standard adopted across thousands of repositories, and in llms.txt for the public web. The knowledge layer is no longer one vendor's bet. It is what the ecosystem converged on.

Six principles emerged in practice. We did not define them upfront. We hit constraints, formalized rules, and they stuck because removing any one of them brought the failures back. What follows walks through the six, then shows what they enable.

THE ARCHITECTURE

## Six rules underneath the architecture

01

### Knowledge is Markdown in Git

The canonical format for everything the company knows is plain Markdown files in a Git repository. Not a database, not a SaaS platform, not a proprietary format. Markdown is human-readable, version-controlled, portable across tools, and natively parseable by every major LLM. The constraint is productive. It forces clarity on what counts as durable knowledge versus what counts as ephemera.

This principle has a corollary that sounds like an engineering opinion but is not: **everyone should use Git.** Lawyers, marketers, finance, customer success, executive assistants. Everyone whose work product is durable enough to be worth keeping should be in a Git repository by next year. An agent that writes against shared knowledge is only as trustworthy as the access pattern underneath it. Versioning, branching, and review-before-merge are the smallest set of mechanics that let an agent change shared truth without anyone losing the ability to roll back, audit, or contest the change. No other access pattern in widespread use today gives you all three. Email does not. SaaS document tools do not. Wikis do not. Trust is downstream of traceability, and traceability is what version control is for.

For non-engineering contributors, Git is the toll gate they hit in week one. Branching, committing, and pulling remain a learning curve. The agent helps: natural-language Git commands, narrated diffs, branch-naming suggestions, all of which reduce the cliff. The cliff is still real, and the tooling will absorb more of it over the next year. Plan for it.

What owning the knowledge layer looks like in practice: a partner sent thousands of external domains pulled from their DNS telemetry, asking which were genuine threats and how serious. In the traditional workflow this is a multi-week analyst job. One product manager in our pilot wrote a skill that orchestrated five separate threat-intelligence sources, ranked the domains by risk tier, and produced a branded PDF, an executive HTML report, and a working Excel file. The partner had it the next morning. Along the way, the same architecture surfaced a productizable service the partner had not asked for. None of the work was a one-off prompt. All of it was reusable infrastructure. The documents an agent produces today are useful but regenerable. The encoded knowledge that shapes how every future document gets produced is the asset that compounds.

## 02 Progressive disclosure

No agent and no human should load the entire knowledge base to answer a question. The system is organized as a thin index at the top, a root instruction file of about fifty lines that points to topic-specific directories, each with its own focused files. The agent reads the index first, then loads only what the current task requires. The repository can grow to thousands of files without slowing the agent down, because at any moment the agent operates on a small, relevant subset. The corollary worth naming: you do not need to know where each piece of information lives. "Where is X?" is the model's job, not the user's.

## 03 Shared truth, personal workspace

There is one source of truth that everyone reads from and contributes to. There is also a personal layer that each contributor manages on their own: notes, drafts, task lists, working memory. The shared layer is governed (reviewed before merge). The personal layer is autonomous (gitignored). This separation prevents the two failure modes of knowledge systems: the wiki nobody updates because contributing is too heavy, and the wiki nobody trusts because anyone can write anything.

THE KNOWLEDGE LAYER: TWO LAYERS, FIVE SURFACES

COMMONS · SHARED · VERSIONED · GOVERNED

### Shared repository

`knowledge-base/`
what the company knows

`shared-projects/`
work in flight

`tools/`
executable capabilities

slow change · reviewed before merge

WORKSHOP · PERSONAL · AUTONOMOUS · PRIVATE

### Personal layer

`memory/`
people, projects, drafts

`CLAUDE.md + TASKS.md`
personal navigation

`.claude/skills/`
procedural workflows

fast change · autonomous · gitignored from shared repo

## 04 Tool agnosticism

The knowledge layer does not depend on any specific AI tool, editor, or platform. Git is the transport layer. Markdown is the storage format. Any LLM agent that can read files and follow instructions can operate on it. Tool-specific configuration exists as a thin adapter on top of the portable base. Switch tools — Claude Code, Cursor, GitHub Copilot, OpenCode, or anything else — and the layer is unchanged. Only the manifest filename differs. The cost of being locked in to any one vendor approaches zero. The cost of being locked out of the convergence is high and rising.

## 05 Status markers as behavioral gates

Documents contain machine-readable signals about their own reliability. `[TBD]` means the agent should not present adjacent content as fact. `[NEEDS-UPDATE]` means flag uncertainty. `[DRAFT]` means do not appear in final outputs. `[CONFIDENTIAL]` means extra care. These markers turn passive documents into active instructions. Constraints travel with the knowledge, not with the chat session or the tool configuration. Once you have markers, the document is no longer a description of state. It is a participant in the workflow.

## 06 Skills as reusable intent

Agent workflows should be encoded as reusable skills, not one-off prompts. A skill is a Markdown file that defines a specific workflow, its trigger, the tools it should use, and its expected output. Skills are version-controlled alongside the knowledge base, shareable across contributors, and composable with each other. A skill for generating a quarterly business review can call a skill for applying brand identity, which can call live data connectors, producing a complete branded document from a single natural-language command. Skills are reusable expressions of how a function thinks. They are how the knowledge layer stops being a reference library and becomes an executable operating system.

WHAT WE BELIEVE IN

# The world model

Today, what enters the knowledge layer is decided by people, often with heavy agent help. The pipe from operational signals into memory is not yet closed. Contributors write, review, merge. The direction we are heading — and the one Block describes as a "world model" — goes further. The same layer fed continuously by the inputs the company already produces. Customer conversations, product usage, financial data, lifecycle events. Updated automatically as those signals arrive. The first half is real today. Skills already pull live operational data into outputs through MCP connectors. The remaining half — the closed loop where those same signals also update memory without a person in the middle — is the next surface we are building. Every principle above already pays off without it.

WHAT THE RULES ENABLE

# The org chart becomes skeuomorphic

Skeuomorphism is the design pattern of making new things look like the old things they replace. Early iOS embraced it heavily: leather stitching in Calendar, wood-grain bookshelves in iBooks, the trash-can icon, a document called *document*. It worked because it borrowed credibility from the analog object. Apple eventually moved past it. Companies have not moved past it with AI. Most are pasting agents on top of processes designed around the org chart. The org chart is the analog object. It is the most expensive piece of skeuomorphism in the company.

The chart says: here is a function, here are the kinds of problems it owns, here are the people allowed to build certain artifacts, here is who calls whom when knowledge needs to move. None of this stays the way the chart drew it once the knowledge layer and the agentic architecture exists.

Beacraft proposes a hierarchy of human work in agentic organizations. **Operators** do work with AI assistance. **Designers** build reusable workflows. **Architects** encode intent and judgment into infrastructure that agents navigate autonomously. The discourse around this hierarchy assumes Architects emerge slowly, as a specialized role, mostly inside engineering. We watched several team members make that shift within their first week. None of them were engineers.

A product manager built the first plugin distributed across the company: six cybersecurity skills, later wired into the live data layer. Marketing built a plugin that runs invisibly — every output comes back already in the company's voice. Customer success built a recurring morning briefing that cross-references calendar, mail, and chat into a daily prioritized day. The investigations team built a skill that runs full investigations from a case ID, queries five data sources, and returns a formatted dossier.

These are not engineers doing engineering. These are PMs doing analyst work, marketers doing platform engineering, customer success doing pre-sales, investigators doing product engineering. The function on the chart did not change. The work that the function does, did. The honest answer at week seven is that the line between functions is moving faster than we expected, and the people best positioned to redraw it are rarely the people we would have guessed.

Roles do not disappear. They become fluid. Once a function's methodology is encoded in the knowledge layer as a reusable skill, every other function can compose against it. The chart stays. The boundaries become permeable. Everybody can do more with agents because everybody can read what every other function knows.

THE ARCHITECT EMERGENCE: ASSUMED VS. OBSERVED

| ASSUMED (PRIOR MODEL) | OBSERVED (OUR PILOT) |
| --- | --- |
| Operator<br>months of practice | Operator + Designer + Architect<br>same person · same week · inside their existing function |
| Designer<br>months more | Six examples across six different functions<br>C-Level, PM, Marketing, CS, Operations, Research, Intelligence |
| Architect<br>years · specialized · inside engineering | None of them were engineers. |
| *Sequential progression, typically years to reach Architect.* | |

## Agents contribute arguments, not just artifacts

Speed is not the most striking thing about this. The most striking thing is the kind of work the system began to do.

Earlier in the experiment, a leadership meeting produced decisions that needed immediate operationalization. The agent processed the transcript, cross-referenced it against the shared knowledge base, ran market research on its own initiative, and concluded that the framing was wrong. The two approaches being debated were already deployed together across the ecosystem, complementary by design. The corrected framing reshaped the executive briefing from presenting a choice to presenting a sequence. Transcript to revised executive briefing, in one working session.

The agent did not just execute. **It argued. And it was right.**

It was not the only time. The agent has produced first drafts of strategic plans, working from the API documentation and the knowledge base alone, that the team described as *very good for a first draft* — the bar for *first draft* had moved up. It has surfaced ambiguity in our own data schemas that humans had not noticed. It has identified incorrect attributions in human-written meeting notes and proposed surgical edits across the affected files to correct the record. None of these are single tasks. They are categories of work where the agent stops being a faster typist and becomes a participant who has read more than anyone else in the room.

This is what hybrid teams look like before anyone calls them that. Not autonomous agents replacing functions. Not humans supervising bots. People doing their actual work, while producing, through that same work, the infrastructure that lets the next agent argue better than the last one.

## Tokens are the new working capital

If you take one operational lesson from this piece, take this one. The cost of running an agent is not the tool subscription. It is the tokens.

The mechanism is not obvious. An instruction's token cost scales with the accumulated context of the session. An instruction that costs twenty tokens in isolation may cost five thousand tokens within a session that has accumulated ten thousand tokens of prior context. Running trivial tasks within a long session is disproportionately expensive relative to running the same task in a fresh one. Most contributors who consume the most tokens are unaware they are doing so. Without per-user visibility, individuals have no way to self-correct.

Pricing is opaque. The default model is usually the most expensive one. The cheaper models handle the majority of the work at the same quality. Most contributors do not know to switch. Live artifacts that re-fetch fresh data on every page open quietly multiply the per-session cost. Long sessions accumulate cost the way idle EC2 instances accumulated cost in 2010. Most companies will figure this out the expensive way.

> **Tokens are working capital.** They require the kind of attention that AWS bills required when cloud started, and the same kind of monitoring infrastructure. The discipline lives closer to the CFO's office than to the CTO's. The companies that compound will treat token budgets as a real line item, with per-team visibility, fallback strategies for vendor outages, and explicit policy on when to use which model. The field report has the operational playbook. Read it before you scale.

## The next phase

These are not predictions. They are observations from week seven of a pilot that started with two people and reached more than twenty across eight functions. We watched eight functions germinate. The first contributor in each function is now positioned to do something different from what they did in week one: not write the next skill alone, but pull the rest of their function up.

Customer success is targeting hyperproductivity that lets the same headcount deliver high-touch attention to the entire portfolio, not just the top of it. Investigations is targeting a state of revising agent-drafted dossiers rather than constructing them from scratch. Marketing is targeting end-to-end content autonomy, where any contributor in any function produces branded, on-message content from a natural-language ask without routing through marketing for review.

This is the operational test of the third claim above: that roles become fluid when the substrate exists. Not just that one Architect emerges in each function, but that **the function as a whole reshapes around what the Architect built.** We do not yet know how this scales past twenty contributors. We will know more by week fourteen.

We are publishing at week seven, not week seventy. The model is evolving. The pattern is forming. Most companies talking about agentic transformation will spend the next two years choosing tools. The companies that compound will spend the next two years building the knowledge layer. We are publishing this to make the gap visible while it is still openable.

If you are running something adjacent inside your company, write back. The people most likely to find what we have missed are the people running the same kind of experiment one country over.

---

*The field report, with the full architecture, the failure modes, the conventions we settled on, and the seven weeks of practice behind this manifesto, is here: An Agentic Operating System: Seven Weeks of Practice. The starter kit, a working seed you can fork to try this on your own team, is here: github.com/Axur-Inc/agentic-os/tree/main/starter-kit.*

AXUR An Infoblox Company

TLP:CLEAR

COMPANION RESOURCES

# The receipts are in the field report.

The full architecture, conventions, failure modes, and seven weeks of practice behind this manifesto.

FIELD REPORT

*An Agentic Operating System: Seven Weeks of Practice*

github.com/Axur-Inc/agentic-os/blob/main/field-report.md

STARTER KIT

A forkable seed of the architecture. Open in any agentic tool and say *set me up*.

github.com/Axur-Inc/agentic-os/tree/main/starter-kit

AUTHORS

Fábio Ramos, CEO
Jônadas Techio, Staff PM



CLASSIFICATION

TLP:CLEAR — unrestricted distribution